Java 25: The Future of Coding, Now

By **durgeshkumar8896** • Wed Sep 17 2025

Java 25 (JDK 25), released on **September 16, productivity**, **performance**, and **clean syntax** er coding to developers. Focused on and powerful than ever.

From **instance main methods** to **scoped values** and **compact source files**, Java 25 redefines how modern applications are built.

1. Instance Main Methods

Now in **Java 25**, You no longer need **public static void main(String[] args)** for simple programs. You can write **main** without **static, public, or even the String[]** args if you don't need them.

Copy

```
class Test{
 void main() {
    System.out.println("Hello, World!");
    System.out.println("This is testing file");

}
}
```

Java application launch follows a strict sequence to find a valid `main` method. The launcher's selection process is as follows:

1. **Priority to Standard Main:** The JVM first looks for a traditional `public static void main(String[] args)` method, which is the most common entry point.

2. **Fallback to**) method (with no pa

3. **Instance M** class. This requires the class to have a **public, no-argument constructor**. The launcher creates an instance using this constructor and then calls the instance's `main` method.

4. **Error Condition:** If no valid `main` method is found, or if an instance `main` exists without a required constructor, the launcher throws an error and terminates.

2. Compact source files

Now in J**ava 25**, You can write a **standalone Java source file** without declaring a class explicitly (no class Hello { ... } needed). The compiler will assume there is an implicit class behind the scenes.

Now, you can create a file with just your methods and variables. The Java compiler will automatically place them inside a hidden, or "unnamed," class for you. This means you can focus on learning the basics of programming without first needing to understand advanced concepts.

How to Write a Compact File
Simply create a `.java` file and write your code directly in it. You **must** have a `main` method, as it's the program's starting point.

Example: Hello, World! Made Simple

Copy

```
void main() {
    System.out.println("Hello, World!");
}
```

You can also use methods and fields you define right in the same file:

Copy

```
String message = "Hello, World!";

void main() {
    printMessage();
}

void printMessage() {
    System.out.println(message);
}
```

Key Things to Know About Unnamed Classes:

- The class is created automatically and is **final** (cannot be extended).
- It has a default, no-argument constructor you can't see.
- You **can** use the `this` keyword inside the file to refer to the current instance.
- The class name is generated by the compiler and is not for you to use in your code.
- Must have a launchable `main` method; if it does not, a compile-time error is reported.

3. Simplifying Java Console Input and Output for Beginners

Writing to and reading from the console is a fundamental first step for new programmers. Traditionally, Java made this more complex than necessary. New features now make basic console interaction much simpler.

Reading user input was even harder. The standard code required understanding advanced concepts like **BufferedReader**, **InputStreamReader**, and handling **IOException** with **try/catch** blocks. This boilerplate code was a significant barrier to writing simple interactive programs.

The New Solution: The **IO** Helper Class
To solve this, Java introduces a new java.lang.IO class. It provides simple, static methods for all basic console operations, eliminating

the need for c

Copy

```
import sta

void main() {
    println("Hello, World!");
}
```

For input

Copy

```
import static java.lang.IO.*;
void main(){
 var name=readln("what is your name?");
 println("Hello, " + name);
}
```

Key Methods of the `IO` Class:

- `IO.print("text")` - Prints text without a new line.
- `IO.println("text")` - Prints text and moves to the next line.
- `IO.println()` - Just prints a blank line.
- `IO.readln()` - Pauses and reads a line of input from the user.
- `IO.readln("Prompt: ")` - Displays a prompt and then reads input.

Since the `IO` class is in the fundamental `java.lang` package, no `import` statement is needed. This works in any Java program, from simple unnamed classes to full traditional applications.

4. Automatic imports of common APIs

In compact source files, classes from commonly used packages in java.base (like java.util, java.io) are automatically available. You don't always have to write import statements for them.

Copy

```
void main(){
    IO.println("Hello, World!");
}
```

5.Flexible Constructor Bodies

Java 25 allows constructor bodies to have statements before calling **super(...)** or **this(...)**. Normally, those calls must be the very first line.

These early statements (called the prologue) cannot use the object under construction (no this or accessing instance methods/fields that are already initialized). But you can initialize uninitialized fields or do safe computations.

Copy

```
class Parent{
    Parent(){
        IO.println("Parent class constructor");
    }

}

class Child extends  Parent{
    Child(){
        IO.println("This is child class constructor");
        super();
```

```
    }
    publi
        ne
    }
}
```

output:

```
This is child class constructor
Parent class constructor
```

Key Rules:

- Statements before the super(...) or this(...) call are called the prologue; those after are the epilogue.
- In the early part (prologue), you cannot use this or any instance methods/fields except to initialize uninitialized fields in that class.
- return statements are allowed in the epilogue (but not with an expression), not in the prologue.
- For record classes, enums, and nested classes, similar rules apply, with some existing constraints still in place.

6. Module Import Declarations

This feature adds a new way to import all packages that a module exports by using a single declaration:

```
import module M;
```

This helps reduce the need to write many import statements for individual packages, especially from modular libraries.

Before this feature:

```
import java.util.*;
import java.sql.*;
import javax.sql.*;
import java.nio.file.*;

public class MyApp {
    // you have to import each needed package separately
    Path path = java.nio.file.Paths.get("file.txt");
    Map<String, String> map = new HashMap<>();
    Connection conn = DriverManager.getConnection(...);
    // and so on
}
```

After (with module import)

```
import module java.base;
import module java.sql;

public class MyApp {
    Path path = Paths.get("file.txt");        // from java.base
    Map<String, String> map = new HashMap<>(); // from java.base
    Connection conn = DriverManager.getConnection(...); // from java.sql
}
```

7. Scoped Values

A scoped value is a new way in Java to share immutable data (data that doesn't change) across methods in a thread and child threads, without passing that data explicitly all the time.

It is meant to be simpler and more efficient than using **ThreadLocal.**

Scoped values

Declaration:

You declare a scoped value (usually static final) of some type.

Copy

```
static final ScopedValue<FrameworkContext> CONTEXT = ScopedValue.newInstance();
```

Binding a value:

Use ScopedValue.where(...).run(...) (or .call(...)) to bind a value for that scope. While inside that run block, methods (direct or indirect) can read the scoped value via .get().

Copy

```
where(CONTEXT, context).run(() -> {
    // inside here, CONTEXT.get() works in this thread & its callees
    Application.handle(request, response);
});
```

Lifetime:

The binding is only valid during the execution of that run block (or call). Once run finishes, the binding is removed.

LoggerExample

Copy

```
import java.lang.ScopedValue;
import static java.lang.ScopedValue.where;

public class LoggerExample {

    // Define a ScopedValue for request ID
    static final ScopedValue<String> REQUEST_ID = ScopedValue.newInstance();

    public static void main(String[] args) {
        handleRequest("REQ-101");
        handleRequest("REQ-202");
    }

    static void handleRequest(String id) {
        // Bind the request ID for this request
        where(REQUEST_ID, id).run(() -> {
            process();
            log("Request completed!");
        });
    }

    static void process() {
        log("Processing data...");
        // Imagine more deep calls here
        saveToDB();
    }

    static void saveToDB() {
        log("Saving to database...");
    }

    static void log(String message) {
        // Access the bound request ID
        System.out.println("[" + REQUEST_ID.get() + "] " + message);
    }
}
```

output:

```
bash-5.1#
[REQ-101]
[REQ-101]
[REQ-101] Request completed!
[REQ-202] Processing data...
[REQ-202] Saving to database...
[REQ-202] Request completed!
```

Conclusion

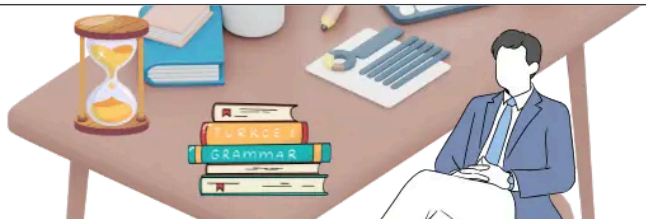
Java 25 isn't just another version — it's a **modern redesign** of how Java is written and taught.
With compact files, flexible constructors, and simple I/O, it's easier for **beginners**, while **scoped values and module imports** make life better for **professionals**.

👉 **Java 25 proves that the future of Java is simpler, faster, and smarter.**

Share this article ...

WhatsApp Facebook LinkedIn X

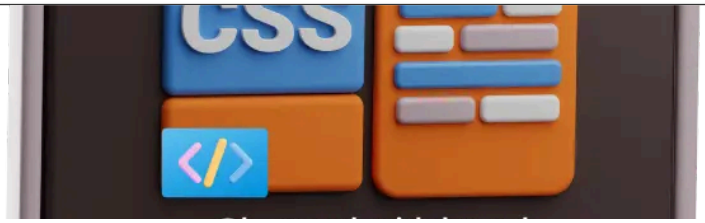
Trending Blogs...



I Used These 3 ChatGPT Prompts Daily in 2025 – Saved 3 Hours

Discover 3 powerful ChatGPT prompts that save you hours daily. Boost your productivity with AI in 2025.

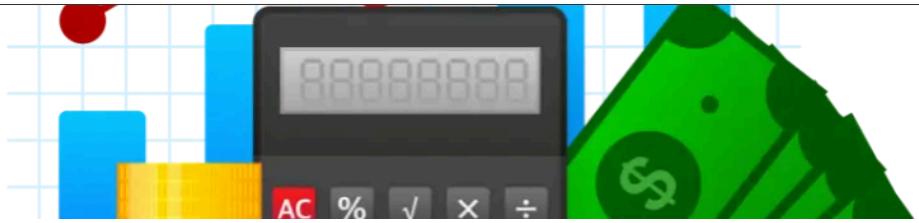
Read More →



Top 10 CSS Tips to Instantly Improve Your Design Skills (Beginner Friendly)

Before writing CSS, know how it connects with HTML and why using external stylesheets is best.

Read More →



Best Investment Apps in India 2025 – Start Investing Smartly from ₹100

Investing today has become easier than ever — thanks to powerful mobile apps that put the entire stock market, mutual funds, and digital assets right in your pocket .

Read More →

Top 10 Highest-Paying Skills to Learn in 2025 (Earn ₹15–50 Lakh/Year)

Hey there! If you're feeling the pressure of keeping up with today's crazy-fast job market, you're not alone. AI, automation, and digital transformation are flipping industries upside down, and the skills that used to land you a solid gig might not cut it anymore.

Read More →

iPhone 17 Series Launch 2025 – India Price, Specs, Pros & Cons

Apple has officially unveiled the **iPhone 17 series** on **September 9, 2025**, at its **"Awe Dropping" event**. The lineup includes **iPhone 17, iPhone 17 Air, iPhone 17 Pro, and iPhone 17 Pro Max**. With **120 Hz displays across all models, ultra-thin design, A19 chips, and camera upgrades**, Apple is raising the bar…

Read More →

Async/Await vs Promises in JavaScript – Which Should You Use in 2025?

Asynchronous programming is one of the most important topics in modern JavaScript development. Whether you're building a web app, mobile application, or integrating APIs, you'll often deal with operations that don't finish instantly, such as fetching data or reading files.

Read More →

Share this article ...

WhatsApp Facebook LinkedIn X

Discover more APIs Coding Computer programming JDK applications Application programming interface Hello, World Application Java Development Kit coding

Discover more

- Process
- applications
- Compiler
- Application software
- Coding
- Application
- coding
- Computer programming
- APIs
- "Hello, World!" program

Learn Code With Durgesh

Offering free & premium coding courses to lakhs of students via YouTube and our platform.

Explore

About Us

Courses

Blog

Contact

FlexBox Game

Legal

Privacy Policy

Terms & Conditions

Refund Policy

Support

Contact

+91-9839466732

support@learncodewithdurgesh.com

Substring Technologies, 633/D/P256 B R Dubey Enclave Dhanwa Deva Road Matiyari Chinhat, Lucknow, UP, INDIA 226028